

BookVerse Database Schema Documentation

1. Overview

BookVerse uses **MongoDB**, a NoSQL document database, accessed through the **Mongoose ODM** (Object Document Mapper). Instead of traditional SQL tables, data is stored in Collections of JSON-like documents.

The database contains three collections:

Collection	Purpose	Documents represent...
users	Stores registered user accounts	One document per user
books	Stores book records added to the system	One document per book
reviews	Stores user-written reviews for books	One document per user-book pair

MongoDB stores each document with an auto-generated `_id` field (`ObjectId`). This acts like a primary key. Mongoose exposes it as `id` (string) in JSON responses.

2. Collections Summary

Collection	Key Fields	Unique Constraints	Timestamps
users	username, passwordHash	username	createdAt, updatedAt
books	ISBN, title, author, content	ISBN	createdAt, updatedAt
reviews	userId, bookId, review_text	(userId + bookId)	createdAt, updatedAt

3. Collection: users

Stores registered user accounts. Each document represents one user. The `passwordHash` field stores a bcrypt hash — the plain-text password is never saved.

Fields

Field	Type	Required	Constraints	Default
_id	ObjectId	Auto	Primary key, auto-generated by MongoDB	—
username	String	Yes	Unique · Lowercase · 3–32 chars · alphanumeric+underscore	—
name	String	No	Max 80 chars · Trimmed	""
email	String	No	Max 120 chars · Lowercase · Trimmed	""
profilePicture	String	No	Max 500 chars · URL string · Trimmed	""
favoriteGenre	String	No	Max 80 chars · Trimmed	""
passwordHash	String	Yes	bcrypt hash · Hidden from JSON output (select: false)	—
tokenVersion	Number	No	Integer · Used to invalidate old refresh tokens	0
createdAt	Date	Auto	Set automatically by Mongoose timestamps	—
updatedAt	Date	Auto	Updated automatically on every save	—

Example Document

```
{
  "_id": "664a1b2c3d4e5f6789012345",
  "username": "john_doe",
  "name": "John Doe",
  "email": "john@example.com",
  "profilePicture": "https://example.com/avatar.jpg",
  "favoriteGenre": "",
  "tokenVersion": 0,
  "createdAt": "2026-05-15T10:00:00.000Z",
  "updatedAt": "2026-05-15T10:00:00.000Z"
}
```

Security note: The `passwordHash` field uses `select: false` in Mongoose, so it is never included in query results unless explicitly requested. Passwords are hashed with bcrypt before storing.

4. Collection: books

Stores book records. Any client can add a book by providing its ISBN, title, and author. Each book has a unique ISBN. The ISBN field is indexed for fast lookups.

Fields

Field	Type	Required	Constraints	Default
<code>_id</code>	ObjectId	Auto	Primary key, auto-generated by MongoDB	—
ISBN	String	Yes	max 20 chars, Unique	—
title	String	Yes	max 200 chars	—
author	String	Yes	max 100 chars	—
content	String	No	No length limit	""
createdAt	Date	Auto	Set automatically by Mongoose timestamps	—
updatedAt	Date	Auto	Updated automatically on every save	—

Example Document

```
{
  "_id": "664b2c3d4e5f6789012346",
  "ISBN": "978-3-16-148410-0",
  "title": "The Great Gatsby",
  "author": "F. Scott Fitzgerald",
  "content": "",
  "createdAt": "2026-05-10T08:00:00.000Z",
  "updatedAt": "2026-05-10T08:00:00.000Z"
}
```

The title and author search endpoints use a case-insensitive regex (partial match). Special characters in search queries are escaped to prevent **ReDoS** attacks.

5. Collection: reviews

Stores user reviews for books. Each user can have at most one review per book — enforced by a compound unique index on `(userId, bookId)`. Submitting a new review on a book you already reviewed will update the existing one (upsert).

Fields

Field	Type	Required	Constraints	Default
<code>_id</code>	ObjectId	Auto	Primary key, auto-generated by MongoDB	—
<code>userId</code>	ObjectId	Yes	References <code>users._id</code> · Indexed	—
<code>bookId</code>	ObjectId	Yes	References <code>books._id</code> · Indexed	—
<code>review_text</code>	String	Yes	1–2000 chars · Trimmed	—
<code>createdAt</code>	Date	Auto	Set automatically by Mongoose timestamps	—
<code>updatedAt</code>	Date	Auto	Updated automatically on every save	—

Example Document

```
{
  "_id": "664c3d4e5f6789012347",
  "userId": "664a1b2c3d4e5f6789012345",
  "bookId": "664b2c3d4e5f6789012346",
  "review_text": "A timeless classic about ambition and the American Dream.",
  "createdAt": "2026-05-14T09:30:00.000Z",
  "updatedAt": "2026-05-14T09:30:00.000Z"
}
```

6. Relationships Diagram

MongoDB is a non-relational (NoSQL) database — there are no foreign key constraints at the database level. Relationships are handled by storing `ObjectId` references in documents and using Mongoose's `populate()` to join them at the application level.

Entity Relationship Overview

Relationship	Type	How it works
users → reviews	One-to-Many	A user can write many reviews. reviews. userId stores the user's _id.
books → reviews	One-to-Many	A book can have many reviews. reviews. bookId stores the book's _id.
users ↔ books (via reviews)	Many-to-Many	A user can review many books; a book can be reviewed by many users.

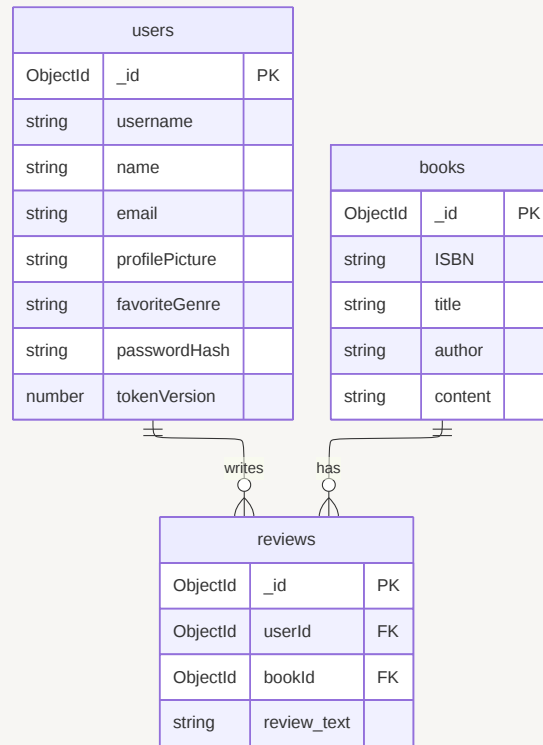
```
erDiagram
    users {
        ObjectId _id PK
        string username
        string name
        string email
        string profilePicture
        string favoriteGenre
        string passwordHash
        number tokenVersion
    }
    books {
        ObjectId _id PK
        string ISBN
        string title
        string author
        string content
    }
    reviews {
        ObjectId _id PK
        ObjectId userId FK
```

```

    ObjectId bookId FK
    string review_text
  }

  users ||--o{ reviews : "writes"
  books ||--o{ reviews : "has"

```



7. Indexes

Indexes make queries faster by letting MongoDB jump directly to relevant documents instead of scanning the entire collection.

Collection	Index	Type	Purpose
users	username	Unique	Ensures no two users share a username. Speeds up login lookups.
reviews	userId	Single field	Speeds up queries that fetch all reviews by a user.
reviews	bookId	Single field	Speeds up queries that fetch all reviews for a book.

Collection	Index	Type	Purpose
reviews	userId + bookId	Unique compound	Enforces the rule: one review per user per book.
books	ISBN	Unique	Ensures no two books share the same ISBN. Speeds up ISBN lookups.
books	title + author	Text	Enables full-text search across title and author fields using MongoDB's \$text operator.

The compound unique index on `reviews (userId, bookId)` is the most important constraint in the schema. It prevents duplicate reviews at the database level, not just the application level — making the system more robust.

8. Design Notes & Constraints

Why MongoDB?

MongoDB is a document database that stores data as JSON-like objects. It is schema-flexible, easy to set up with Node.js (via Mongoose), and well-suited for rapidly evolving data structures like book metadata.

Mongoose ODM

Mongoose adds a schema layer on top of MongoDB, letting us define field types, validations, defaults, and relationships in JavaScript. It also provides methods like `find()`, `findById()`, `create()`, and `save()`.

Timestamps

All three collections use Mongoose's `timestamps: true` option, which automatically adds `createdAt` and `updatedAt` fields to every document. No manual date management needed.

Password Security

Plain-text passwords are never stored. The `passwordHash` field holds a bcrypt hash. The field is marked `select: false` so it is excluded from query results by default, preventing accidental exposure in API responses.

tokenVersion

The `tokenVersion` field on users is an integer counter used to invalidate all existing refresh tokens for a user (e.g. on logout from all devices). When the version increments, old tokens with a lower version are rejected.

ISBN is unique

The `books` collection enforces uniqueness on ISBN. Each book has its own ISBN.

Input Sanitisation

Title and author search queries are passed through a `safeRegex()` helper that escapes special regex characters and limits input length to 80 characters. This prevents **ReDoS** (Regular Expression Denial of Service) attacks.